COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010/DATA STRUCTURE AND ALGORITHMS	**SECTION:**	CPE21S2
DATE PERFORMED:	05/08/2026	**DATE SUBMITTED:**	07/08/2026
NAME:	Bungag, Marc Jhowell M.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HOA 5.1 Queues

1. PROCEDURE OUTPUT

What procedures did you perform? (**PROGRESS REPORT**)

SCREENSHOT

```
ILOA > ILOAmain.cpp > main()
#include<iostream>
#include<queue>

void Display(std::queue<char> copyQ){
    //create a copy of the queue
    std::queue<char> temp(copyQ);
    //loop until empty
    while(!temp.empty()){
        std::cout<<temp.front()<<" "<<std::endl;
        //dequeue the front
        temp.pop();
    }
        //display the front
        std::cout<<"Current Front is: "<<copyQ.front()<<std::endl;
    //add a new line
    std::cout<<std::endl;
}

int main(){
    //create an object:
    std::queue<char>myQ;

    //use the enqueue operation
    myQ.push('M');
    myQ.push('A');
    myQ.push('R');
    myQ.push('C');
    std::cout<<"Current Front is: "<<myQ.front()<<std::endl;
    std::cout<<"Current Back is: "<<myQ.back()<<std::endl;

    //prototyping
    Display(myQ);

    //use the dequeue operation
    myQ.pop();
    Display(myQ);
    myQ.pop();
    Display(myQ);
    myQ.pop();
```

OUTPUT

```
Current Front is: M
Current Back is: C
M
A
R
C
Current Front is: M

A
R
C
Current Front is: A

R
C
Current Front is: R

C
Current Front is: C

Is the Queue Empty? 1
```

```
33
34      //use the dequeue operation
35      myQ.pop();
36      Display(myQ);
37      myQ.pop();
38      Display(myQ);
39      myQ.pop();
40      Display(myQ);
41      myQ.pop();
42
43      std::cout<<"Is the Queue Empty? "<<myQ.empty()<<std::endl;
44
45
46
47
48
49
50      return 0;
51
52  }
```

ILOA

OBSERVATION:

In this code, we demonstrate how a queue works. The characters “M”, “A”, “R”, and “C” are inserted into the queue using push(), making M the front and C the back. The Display() function creates a copy of the queue so that its contents can be printed without modifying the original queue. Each call to pop() removes only the front element, showing that the earliest inserted element is always removed first. As the program continues dequeuing, the front changes from “M” to “A” to “R” to “C” until the queue becomes empty. Finally, the empty() is a boolean function that confirms the queue no longer contains any elements by returning 1.

SCREENSHOT	OUTPUT
.H FILE	Testing Enqueue Operator Front: M Back: M Front: M Back: A Front: M Back: R Front: M Back: C Displaying Queue MARC Testing the Dequeue Operator Front: A Back: C Front: R Back: C Front: C Back: C The Queue is Empty Testing if the Queue is Empty The Queue is Empty

```cpp
void Dequeue (qNode<T>** frontPtr, qNode<T>** backPtr){
    //check if the queue is empty
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        std::cout<<"The Queue is Empty"<<std::endl;
        return;
    }
    // create a temporaray variable to store the node to be deleted
    qNode<T>* deleteTemp = nullptr;
    //assign the current fron to the deleteTmep
    deleteTemp = (*frontPtr);
    //check if the queue has only 1 item
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        //means only 1 item is inside the queue
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteTemp;
        return;
    }
    (*frontPtr) = (*frontPtr)->next;
    deleteTemp->next = nullptr;
    //delete the node
    delete deleteTemp;
}

//display all
template <typename T>
void DisplayQ(qNode<T>* frontPtr){
    if(frontPtr == nullptr){
        std::cout<<"The Queue Does Not Exist"<<std::endl;
    }
    while(frontPtr != nullptr){
        std::cout<<frontPtr->data;
        frontPtr = frontPtr->next;
    }
}
```

```cpp
//isEmpty
template <typename T>
bool isEmptyQ(qNode<T>* frontPtr, qNode<T>* backPtr){
    if(frontPtr == nullptr && backPtr == nullptr){
        std::cout<<"The Queue is Empty"<<std::endl;
        return true;
    }
    return false;
}
#endif
```

.CPP FILE

ILOB > ILOBmain.cpp > main()

```cpp
#include <iostream>
#include "queue11.h"

int main (){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    std::cout<<"Testing Enqueue Operator \n";
    Enqueue('M', &front, &back);
    std::cout<<"Front: "<<front->data<<" "<<"Back: "<<back->data<<std::endl;
    Enqueue('A', &front, &back);
    std::cout<<"Front: "<<front->data<<" "<<"Back: "<<back->data<<std::endl;
    Enqueue('R', &front, &back);
    std::cout<<"Front: "<<front->data<<" "<<"Back: "<<back->data<<std::endl;
    Enqueue('C', &front, &back);
    std::cout<<"Front: "<<front->data<<" "<<"Back: "<<back->data<<std::endl;
    std::cout<<std::endl;

    std::cout<<"Displaying Queue"<<std::endl;
    DisplayQ(front);
    std::cout<<std::endl;
    std::cout<<std::endl;

    std::cout<<"Testing the Dequeue Operator \n";
    Dequeue(&front, &back);
    std::cout<<"Front: "<<front->data<<" "<<"Back: "<<back->data<<std::endl;
    Dequeue(&front, &back);
    std::cout<<"Front: "<<front->data<<" "<<"Back: "<<back->data<<std::endl;
    Dequeue(&front, &back);
    std::cout<<"Front: "<<front->data<<" "<<"Back: "<<back->data<<std::endl;
    Dequeue(&front, &back);
    Dequeue(&front, &back);
    std::cout<<std::endl;
    std::cout<<std::endl;

    std::cout<<"Testing if the Queue is Empty"<<std::endl;
    isEmptyQ(front, back);

    return 0;
}
```

ILOB1

OBSERVATIONS:

The program demonstrates the implementation of a queue using a linked list, where each node stores a data value and a pointer to the next node. The Enqueue() function inserts new elements at the rear of the queue, while the Dequeue() function removes elements from the front of the list. As the characters “M”, “A”, “R”, and “C” are enqueued, the front remains M while the back changes to the most recently inserted character. When the dequeue operation is performed, the front updates from “M” to “A” to “R” to “C”, showing that elements are removed in the same order they were inserted. After all elements have been removed, attempting another dequeue correctly displays that the queue is empty as seen in the output, and the boolean isEmptyQ() function confirms that no nodes remain in the linked-list queue. Overall, the implementation shows how a linked list provides a dynamic queue structure without requiring a fixed-size array, allowing nodes to be added and removed efficiently.

SCREENSHOT	OUTPUT
.H FILE	

ILOB > C queueArr.h > ...

```h
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>

//Array-Based Circular Queue
template <typename T>
class queueArr{
    private:
    //pointer to dynamically allocate array
    T* q_array;
    //maximum number of elements a queue can hold
    size_t q_capacity;
    //current number of the elements in the queue
    size_t q_size;
    //Index of the front element
    int q_front;
    //Index of the back element
    int q_back;

    public:
        queueArr(size_t capacity = 10);
        queueArr(const queueArr& other);
        queueArr& operator=(const queueArr& other);
        ~queueArr();
        //queue operations:
        bool Empty();
        bool Full();
        size_t Size();
        void Clear();
        T Front();
        T Back();
        void Enqueue(T value);
        T Dequeue();
};

//contructor
template <typename T>
queueArr<T>::queueArr(size_t MAXcap){
    q_capacity = MAXcap;
    q_size = 0;
    q_front = 0;
    q_back = -1;
    q_array = new T[q_capacity];
}

```

```
Front: 10
Back: 10
Size: 1

Front: 10
Back: 20
Size: 2

Front: 10
Back: 30
Size: 3

Front: 10
Back: 40
Size: 4

Front: 10
Back: 50
Size: 5
Queue is Full

Removed: 0
Removed: 0

Is the Queue Empty? Yes
```

```cpp
//copy contructor
template <typename T>
queueArr<T>::queueArr(const queueArr<T>& other){
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;

    for(size_t i = 0; i < q_capacity; i++){
        q_array[i] = other.q_array[i];
    }
}
//copy assignment operator
template <typename T>
queueArr<T>& queueArr<T>::operator=(const queueArr<T>& other) {

    if (this != &other) {

        delete[] q_array;

        q_capacity = other.q_capacity;
        q_size = other.q_size;
        q_front = other.q_front;
        q_back = other.q_back;
        for (size_t i = 0; i < q_capacity; i++) {
            q_array[i] = other.q_array[i];
        }
    }
    return *this;
}
//destructor
template <typename T>
queueArr<T>::~queueArr(){
    delete[] q_array;
}

//empty
template <typename T>
bool queueArr<T>::Empty(){
    return q_size == 0;
}

//Full
template <typename T>
bool queueArr<T>::Full(){
    return q_size == q_capacity;
}

```

```cpp
//size_T
template <typename T>
size_t queueArr<T>::Size(){
    return q_size;
}

//front
template <typename T>
T queueArr<T>::Front(){
    //check if queue is empty
    if(Empty()){
        std::cout<<"Queue is Empty\n";
        return T();
    }
    //return the fornt
    return q_array[q_front];
}

//back
template <typename T>
T queueArr<T>::Back(){
    if(Empty()){···
    return q_array[q_back];
}

//clear
template <typename T>
void queueArr<T>::Clear(){

    //reset the q_size, q_front, q_back
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

//Enqueue
template <typename T>
void queueArr<T>::Enqueue(T value){

    //check if the queue is full
    if(Full()){
        std::cout<<"Queue is Full\n";
        return;
    }
    //move q_back circularly
    q_back = (q_back + 1) % q_capacity;
    //store the value to the back
    q_array[q_back] = value;
```

```
141     //move q_back circularly
142     q_back = (q_back + 1) % q_capacity;
143     //store the value to the back
144     q_array[q_back] = value;
145     //increment the q_size
146     q_size++;
147 }
148 
149 //Dequeue
150 template <typename T>
151 T queueArr<T>::Dequeue(){
152 
153     //check if the queue is empty
154     if(Empty()){
155         std::cout<<"Queue is Empty\n";
156         return T();
157     }
158     //create a temp variable to store the current front
159     T temp = q_array[q_front];
160     //move q_front circularly
161     q_front = (q_front + 1) % q_capacity;
162     //decrement the q_size
163     q_size--;
164     //reset the indexes if the queue becomes empty
165     if(Empty()) Clear();
166     //return the temporary variable
167     return T();
168 }
169 #endif
```

MAIN.CPP

ILOB > ILOBARRmain.cpp > ...

```cpp
#include <iostream>
#include "queueArr.h"

int main(){
    //creating an object
    queueArr<int> q(5);

    // enqueue 10, 20, 30, 40, 50
    q.Enqueue(10);
    std::cout<<"Front: "<<q.Front()<<std::endl;
    std::cout<<"Back: "<<q.Back()<<std::endl;
    std::cout<<"Size: "<<q.Size()<<std::endl;
    std::cout<<std::endl;
    q.Enqueue(20);
    std::cout<<"Front: "<<q.Front()<<std::endl;
    std::cout<<"Back: "<<q.Back()<<std::endl;
    std::cout<<"Size: "<<q.Size()<<std::endl;
    std::cout<<std::endl;
    q.Enqueue(30);
    std::cout<<"Front: "<<q.Front()<<std::endl;
    std::cout<<"Back: "<<q.Back()<<std::endl;
    std::cout<<"Size: "<<q.Size()<<std::endl;
    std::cout<<std::endl;
    q.Enqueue(40);
    std::cout<<"Front: "<<q.Front()<<std::endl;
    std::cout<<"Back: "<<q.Back()<<std::endl;
    std::cout<<"Size: "<<q.Size()<<std::endl;
    std::cout<<std::endl;
    q.Enqueue(50);
    std::cout<<"Front: "<<q.Front()<<std::endl;
    std::cout<<"Back: "<<q.Back()<<std::endl;
    std::cout<<"Size: "<<q.Size()<<std::endl;
    q.Enqueue(60);
    std::cout<<std::endl;

    std::cout<<"Removed: "<<q.Dequeue()<<std::endl;
    std::cout<<"Removed: "<<q.Dequeue()<<std::endl;
    std::cout<<std::endl;

    q.Clear();
    std::cout << "Is the Queue Empty? "<<(q.Empty()?"Yes" : "No")<<std::endl;
    return 0;
}
```

ILOB2

OBSERVATIONS:

I observed that this program uses a circular queue with a dynamically allocated array and follows the FIFO (First In, First Out) principle. I learned that Enqueue() adds elements at the back while Dequeue() removes elements from the front. The q_front and q_back indexes move circularly using the modulo operator, allowing the queue to reuse available spaces. I also observed that the program checks if the queue is empty or full before performing operations. When I tried to add 60 after adding five elements, the queue displayed "Queue is Full" because it had already reached its capacity. After removing two elements and using Clear(), the queue became empty.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

COMPUTER ENGINEERING

QUEZON CITY

Copy and paste the questions from the lab and include your answers after.

CODE	EXPLANATION

JOBLL.H FILE
`#ifndef JOBQUEUELL`
`#define JOBQUEUELL`
`#include <iostream>`
`#include <string>`
 	
`//Job class`
`class Job{`
`private:`
`std::string Username;`
`int JobID;`
`int NumofPages;`
 	
`public:`
`Job(int ID = 0, std::string Name = "", int Pages = 0){`
`JobID = ID;`
`Username = Name;`
`NumofPages = Pages;`
`}`
`void DisplayCredentials(){`
`std::cout<<"Job ID: "<<JobID<<std::endl;`
`std::cout<<"Username: "<<Username<<std::endl;`
`std::cout<<"Pages: "<<NumofPages<<std::endl;`
`}`
`};`
	In this part of the code, the Job class allows me to store important information about a printing job, such as the Job ID, username, and number of pages. I learned that the constructor initializes the information when I create a job, while the DisplayCredentials() function allows me to display the details. I also understood that making the variables private helps me protect the data from being directly accessed outside the class.

JOBLL.H FILE

```
//create a class node
template <typename T>
class qNode{
    public:
        T data;
        qNode* next;
};
//new node creator
template <typename T>
qNode<T>* new_node(T newData){
    //allocate a space for the new node
    qNode<T>* newNode = new qNode<T>;
    //store the newdata to the newNode
    newNode->data = newData;
    //point the newNode to the next
    newNode->next = nullptr;
    return newNode;
}

// enqueue
template <typename T>
void Enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    //create a new node
    qNode<T>* newNode = new_node(newData);
    //check is the queue is empty
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
        return;
    }
    //if not then insert at the back
    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}
```

```
//dequeue
template <typename T>
void Dequeue (qNode<T>** frontPtr, qNode<T>** backPtr){
    //check if the queue is empty
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        std::cout<<"The Queue in the Printer is Empty"<<std::endl;
        return;
    }
    // create a temporaray variable to store the node to be deleted
    qNode<T>* deleteTemp = nullptr;
    //assign the current fron to the deleteTmep
    deleteTemp = (*frontPtr);
    //check if the queue has only 1 item
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        //means only 1 item is inside the queue
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteTemp;
        return;
    }
    (*frontPtr) = (*frontPtr)->next;
    deleteTemp->next = nullptr;
    //delete the node
    delete deleteTemp;
}

//display all
template <typename T>
void DisplayQ(qNode<T>* frontPtr){
    if(frontPtr == nullptr){
        std::cout<<"The Queue in the Printer Does Not Exist"<<std::endl;
    }
    while(frontPtr != nullptr){
        std::cout<<frontPtr->data;
        frontPtr = frontPtr->next;
    }
}

//isEmpty
template <typename T>
bool isEmptyQ(qNode<T>* frontPtr, qNode<T>* backPtr){
    if(frontPtr == nullptr && backPtr == nullptr){
        std::cout<<"The Queue in the Printer is Empty"<<std::endl;
        return true;
    }
    return false;
}
#endif
```

I reuse the header file of ILOB1 since I am using a linked list to create my printer queue. **(Defend your choice of internal representation: Why did you use arrays or linked lists?)** In answering this question, I think using a linked list is better for the printer simulation because it can dynamically store jobs without setting a fixed limit on the number of jobs. It also makes adding and removing jobs easier since I do not need to shift other elements like I would with an array. This makes the linked list more flexible and suitable for a printer queue where jobs can continuously be added and processed. Additionally, I understand linked lists much better than arrays. I also don't want to allocate my time when coding the circular array because I have to understand it better if I will be using arrays.

PRINTER.H FILE

ILOC > C PRINTER.h > Printer > DisplayJobPrint()

```
#ifndef PRINTER_H
#define PRINTER_H

#include <iostream>
#include "JOBLL.h"

class Printer{
private:
    qNode<Job>* front;
    qNode<Job>* back;

public:
    //Constructor
    Printer(){
        front = nullptr;
        back = nullptr;
    }

    //Destructor
    ~Printer(){
        while(!isEmptyQ(front, back)){
            Dequeue(&front, &back);
        }
    }

    //Add a new job
    void AddJobPrint(Job newJob){
        Enqueue(newJob, &front, &back);
        std::cout<<"Job has been Added to the Printer"<<std::endl;
    }

    //Process one job
    void ProcessJobPrint(){
        if(isEmptyQ(front, back)){
            std::cout<<"There are No Jobs to Process"<<std::endl;
            return;
        }
        std::cout<<"\nProcessing Job..."<<std::endl;
        front->data.DisplayCredentials();
        Dequeue(&front, &back);
        std::cout<<"Job Print Completed!"<<std::endl;
    }

    //Display all pending jobs
    void DisplayJobPrint() {
        if(isEmptyQ(front, back)){
            std::cout<<"There are No Jobs to Process"<<std::endl;
            return;
        }
        qNode<Job>* current = front;
        std::cout<<"\n===== PENDING JOBS ====="<<std::endl;
        while(current != nullptr){
            current->data.DisplayCredentials();
            std::cout<< "~~~~~~~~~~~~~~~~~~~~~~~~~~~~~"<<std::endl;
            current = current->next;
        }
    }
};
#endif
```

In this program, the Printer class is used to manage the printing jobs in a queue. I learned that the constructor initializes the front and back pointers as nullptr, while the destructor removes all remaining jobs when the printer object is deleted. I also understood that AddJobPrint() adds a new job to the queue, ProcessJobPrint() processes one job at a time, and ProcessAllPrint() processes all pending jobs. Lastly, DisplayJobPrint() allows me to view all the jobs that are currently waiting to be processed.

MAIN.CPP

The main function allows me to simulate a shared printer by letting me enter the number of jobs and their details. I can provide the **Job ID, username, and number of pages** for each job, which are then stored in a Job object and added to the printer queue. After entering all the jobs, I can view the pending jobs using DisplayJobPrint(). The program then processes all the jobs one by one using ProcessAllPrint(), allowing me to see how a queue manages printing tasks in a **First In, First Out (FIFO)** order.

HOA5P1 > ILOC > ILOCmain.cpp > main()

```cpp
#include <iostream>
#include <string>
#include "Printer.h"

int main(){
    int ID;
    std::string Name;
    int Pages;
    Printer printer;
    int NumberOfJobs;

    std::cout<<"===== SHARED PRINTER SIMULATION =====">>std::endl;
    std::cout<<"Enter Number of Jobs to be Printed: ";
    std::cin>>NumberOfJobs;

    for(int i = 0; i < NumberOfJobs; i++){
        std::cout<<"\nEnter Details for Job "<<i + 1<<std::endl;

        std::cout<<"Job ID: ";
        std::cin>>ID;

        std::cout<<"Username: ";
        std::cin>>Name;

        std::cout << "Number of Pages: ";
        std::cin >> Pages;

        Job newJob(ID, Name, Pages);
        printer.AddJobPrint(newJob);
    }
    printer.DisplayJobPrint();
    std::cout << "\n===== PROCESSING JOBS =====">> std::endl;
    printer.ProcessAllPrint();

    return 0;
}
```

CONCLUSION:

In this activity, I learned how queues work and how to access them. Queues work in a different principle which is the FIFO (First in, First out) order. Unlike the stacks, whichever is the first one to be in a queue, it is usually the first one to be removed or served. Queues can also be in array or linked lists form. In linked lists, it is easy to dynamically allocate elements and it does not require to shift elements in order to add or remove an element in the queue. In arrays, it needs to be in a circular form in order to be dynamic. It also can store a specific number of elements in a queue. In the supplementary activity, I learned how printers in an office station works using queues in linked lists form. I did great in linked lists , however I had a little trouble understanding arrays because it is hard for me to comprehend.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or

completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.